

Scope & Hoisting — Scope এবং Hoisting

Chapter 6-এ আমরা **Functions** শিখেছি। এখন JavaScript-এর খুব গুরুত্বপূর্ণ একটি বিষয় হলো **Scope** এবং **Hoisting**।

এই chapter ভালোভাবে বুঝলে পরবর্তীতে **Closures, Modules, Async JavaScript, React, Node.js**—সবকিছু বুঝতে অনেক সহজ হবে।

PART 01 — CORE SCOPES: GLOBAL, FUNCTION & BLOCK SCOPES (7.1 – 7.10)

7.1 Scope কী?

Scope মানে হলো—কোন variable বা function-কে কোথা থেকে access করা যাবে, সেই সীমা বা area।

সহজভাবে:

Core Rule: Scope = কোন জায়গা থেকে কোন variable ব্যবহার করা যাবে।

REAL-LIFE EXAMPLE

ধরো একটি বিশ্ববিদ্যালয়ে:

- University-এর main gate → সবাই access করতে পারে
- Department-এর room → শুধু department-এর মানুষ
- Teacher's personal room → আরও limited access

JavaScript-এও variable-এর access একইভাবে সীমাবদ্ধ হতে পারে।

7.2 JavaScript-এ Scope-এর প্রধান ধরন

JavaScript-এ আমরা প্রধানত এগুলো দেখতে পাই:

- Global Scope
- Function Scope
- Block Scope
- Lexical Scope
- Module Scope

7.3 Global Scope

যে variable কোনো function বা block-এর বাইরে declare করা হয়, সেটি সাধারণত **Global Scope**-এ থাকে।

CODE

JAVASCRIPT

CODE

```
let name = "Shariar";

function showName() {
  console.log(name);
}

showName();
console.log(name);
```

OUTPUT

Shariar Shariar

Explanation

JAVASCRIPT

EXPLANATION

```
let name = "Shariar";
```

এটি function-এর বাইরে আছে।

তাই function-এর ভেতর থেকেও:

JAVASCRIPT

EXPLANATION

```
console.log(name);
```

access করা যাচ্ছে।

REAL-LIFE EXAMPLE

University-এর notice board-এ যদি কোনো notice দেওয়া থাকে, তাহলে অনেক department-এর মানুষ সেটা দেখতে পারে।

ঠিক তেমনি global variable অনেক জায়গা থেকে accessible হতে পারে।

7.4 Global Variable-এর সমস্যা

Global variable বেশি ব্যবহার করা ভালো practice নয়।

JAVASCRIPT

CODE

```
let username = "Ripon";

function changeName() {
  username = "Shariar";
}

changeName();

console.log(username);
```

OUTPUT

Shariar

Function-এর ভিতর থেকে global variable পরিবর্তন হয়ে গেছে।

বড় application-এ এমন অনেক global variable থাকলে debugging কঠিন হয়ে যেতে পারে।

7.5 Function Scope

কোনো variable যদি function-এর ভিতরে declare করা হয়, তাহলে সেটি সাধারণত সেই function-এর মধ্যেই accessible।

JAVASCRIPT

CODE

```
function test() {  
  let message = "Hello JavaScript";  
  
  console.log(message);  
}  
  
test();
```

OUTPUT

Hello JavaScript

কিন্তু:

JAVASCRIPT

CODE

```
function test() {  
  let message = "Hello JavaScript";  
}  
  
test();  
  
console.log(message);
```

OUTPUT

ReferenceError

কারণ `message` function-এর বাইরে accessible নয়।

7.6 `var` এবং Function Scope

`var` হলো function-scoped।

JAVASCRIPT

CODE

```
function test() {  
  var x = 10;  
  
  if (true) {  
    var x = 20;  
  }  
  
  console.log(x);  
}  
  
test();
```

OUTPUT

20

কারণ `var` block scope মানে না।

7.7 Block Scope

`{ }` দ্বারা তৈরি block-এর ভিতরে থাকা scope-কে **Block Scope** বলা হয়।

উদাহরণ:

JAVASCRIPT

CODE

```
if (true) {  
  let message = "Hello";  
  console.log(message);  
}
```

OUTPUT

Hello

কিন্তু:

JAVASCRIPT

CODE

```
if (true) {  
  let message = "Hello";  
}  
  
console.log(message);
```

OUTPUT

ReferenceError

কারণ `message` শুধু ওই block-এর ভিতরে accessible।

7.8 `let` এবং `const` হলো Block Scoped

JAVASCRIPT

CODE

```
if (true) {  
  let a = 10;  
  const b = 20;  
  
  console.log(a);  
  console.log(b);  
}
```

OUTPUT

10 20

কিন্তু বাইরে:

JAVASCRIPT

CODE

```
console.log(a);  
console.log(b);
```

access করা যাবে না।

7.9 var VS let VS const

Feature	var	let	const
Scope	Function	Block	Block
Reassign	✓	✓	✗
Redeclare same scope	✓	✗	✗
Hoisted	✓	✓*	✓*
TDZ	✗	✓	✓

* এখানে hoisting হয়, কিন্তু `let` / `const` declaration initialization-এর আগে access করা যায় না।
এটা আমরা একটু পরেই বিস্তারিত দেখব।

7.10 Block Scope-এর Real-life Example

ধরো একটি company-তে:

```
Company
├── HR Department
├── Development Department
└── Marketing Department
```

Development department-এর private variable Marketing department সরাসরি ব্যবহার করতে পারবে না।

JavaScript:

```
JAVASCRIPT
{
  let developer = "Ripon";
}

console.log(developer);
```

CODE

এখানে `developer` block-এর বাইরে পাওয়া যাবে না।

7.11 Nested Scope

একটি scope-এর ভিতরে আরেকটি scope থাকতে পারে।

JAVASCRIPT

CODE

```
let university = "DIU";

function department() {

  let dept = "CSE";

  function student() {

    let studentName = "Ripon";

    console.log(university);
    console.log(dept);
    console.log(studentName);
  }

  student();
}

department();
```

OUTPUT

```
DIU CSE Ripon
```

এখানে:

```
Global Scope
  ↓
department Scope
  ↓
student Scope
```

7.12 Scope Chain

JavaScript কোনো variable খুঁজলে প্রথমে **বর্তমান scope**-এ খোঁজে।

না পেলে outer scope-এ যায়।

আবার না পেলে আরও outer scope-এ যায়।

এটাকে বলা হয়:

Core Rule: Scope Chain

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let university = "DIU";

function department() {

    let dept = "CSE";

    function student() {

        let name = "Ripon";

        console.log(name);
        console.log(dept);
        console.log(university);
    }

    student();
}

department();
```

JavaScript `name` খুঁজবে:

```
student scope
  ↓
department scope
  ↓
global scope
```

7.13 Scope Chain Real-life Example

ধরো তুমি তোমার room-এ একটি mobile খুঁজছো।

প্রথমে:

নিজের Room

না পেলে:

বাসার অন্য জায়গা

তারপর:

অন্য জায়গা

JavaScript variable খোঁজার ক্ষেত্রেও একই ধরনের ধারণা কাজ করে।

7.14 Inner Scope → Outer Scope Access

Inner scope outer variable access করতে পারে।

JAVASCRIPT

CODE

```
let x = 10;

function test() {

  let y = 20;

  console.log(x);
  console.log(y);
}

test();
```

OUTPUT

```
10 20
```

কিন্তু outer scope inner variable access করতে পারে না।

JAVASCRIPT

CODE

```
function test() {
  let y = 20;
}

test();

console.log(y);
```

OUTPUT

```
ReferenceError
```

7.15 Lexical Scope

JavaScript-এর scope **code কোথায় লেখা হয়েছে** তার ওপর নির্ভর করে।

এটাকে বলা হয়:

Core Rule: Lexical Scope

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let name = "Ripon";

function outer() {

  let university = "DIU";

  function inner() {
    console.log(name);
    console.log(university);
  }

  inner();
}

outer();
```

`inner()` function যেখানে লেখা হয়েছে, সেই জায়গার lexical environment অনুযায়ী সে outer variable access করতে পারে।

7.16 Important: Function কোথায় Call করা হচ্ছে সেটা নয়

JavaScript lexical scope follow করে।

JAVASCRIPT

CODE

```
let x = "Global";

function show() {
  console.log(x);
}

function test() {
  let x = "Local";
  show();
}

test();
```

OUTPUT

Global

অনেকে ভাবতে পারে:

```
test()
  ↓
show()
```

তাই `show()` local `x` পাবে।

কিন্তু তা নয়।

`show()` কোথায় **defined** হয়েছে সেটাই গুরুত্বপূর্ণ।

7.17 Shadowing

Outer scope এবং inner scope-এ একই নামের variable থাকলে inner variable outer variable-কে **shadow** করতে পারে।

JAVASCRIPT

CODE

```
let name = "Global";

function test() {
  let name = "Local";

  console.log(name);
}

test();

console.log(name);
```

OUTPUT

Local Global

কী হলো?

Function-এর ভিতরের:

JAVASCRIPT

কী হলো?

```
let name = "Local";
```

outer:

JAVASCRIPT

কী হলো?

```
let name = "Global";
```

কে shadow করেছে।

7.18 Real-life Shadowing Example

ধরো:

```
University Name = DIU
Department Name = CSE
```

কোনো department-এর ভিতরে আবার:

```
Department Name = SWE
```

Department-এর ভিতরে জিজ্ঞেস করলে local নাম পাওয়া যাবে।

7.19 Illegal Shadowing

`let` এবং `var` একসাথে কিছু scope situation-এ conflict করতে পারে।

JAVASCRIPT

CODE

```
let x = 10;

{
  var x = 20;
}
```

এটি error দেবে।

কারণ `var` block scope মানে না এবং একই scope binding conflict তৈরি করে।

PART 03 — HOISTING MECHANICS & TEMPORAL DEAD ZONE (TDZ) (7.20 – 7.30)

7.20 Hoisting কী?

এখন আসি JavaScript-এর খুব famous concept:

🚀 Hoisting

সহজ ভাষায়:

Core Rule: JavaScript execution শুরু হওয়ার আগে কিছু declaration-এর জন্য memory setup করে রাখে।

তাই কিছু ক্ষেত্রে declaration-এর আগে code লিখলেও JavaScript error না দিয়ে কাজ করতে পারে।

7.21 `var` Hoisting

দেখো:

JAVASCRIPT

CODE

```
console.log(x);

var x = 10;
```

OUTPUT

undefined

এখানে error হয়নি।

Conceptually JavaScript এটাকে এমনভাবে দেখে:

JAVASCRIPT

CODE

```
var x;

console.log(x);

x = 10;
```

তাই প্রথমে `x` -এর value:

undefined

7.22 let Hoisting

এবার:

JAVASCRIPT

CODE

```
console.log(x);  
  
let x = 10;
```

OUTPUT

ReferenceError

অনেকে বলে:

Core Rule: "let hoist হয় না!"

এটি পুরোপুরি accurate নয়।

let declaration-ও execution context তৈরি হওয়ার সময় binding পায়, কিন্তু initialization-এর আগে access করা যায় না।

এই সময়টাকে বলা হয়:

Temporal Dead Zone — TDZ

7.23 Temporal Dead Zone — TDZ

TDZ হলো:

Core Rule: let বা const declaration শুরু হওয়ার আগের সেই সময়, যখন variable-কে access করলে error হয়।

EXAMPLE

JAVASCRIPT

EXAMPLE

```
console.log(name);  
  
let name = "Ripon";
```

name declaration-এর আগে access করা হয়েছে।

তাই:

ReferenceError

7.24 TDZ Visual

```
let name = "Ripon";  
  
  ↑  
  |  
Declaration
```

Declaration-এর আগে:

TDZ

Declaration-এর পরে:

Accessible

7.25 `const` এবং Hoisting

JAVASCRIPT

CODE

```
console.log(age);  
  
const age = 23;
```

OUTPUT

ReferenceError

কারণ `const` -ও TDZ-এর মধ্যে থাকে declaration-এর আগে।

7.26 Function Declaration Hoisting

Function declaration hoisted হয়।

JAVASCRIPT

CODE

```
sayHello();  
  
function sayHello() {  
  console.log("Hello!");  
}
```

OUTPUT

Hello!

এটি কাজ করে।

7.27 Function Expression Hoisting

কিন্তু:

JAVASCRIPT

CODE

```
sayHello();  
  
const sayHello = function () {  
  console.log("Hello!");  
};
```

OUTPUT

ReferenceError

কারণ এখানে `sayHello` একটি `const` variable-এর মধ্যে function রাখা হয়েছে।

7.28 var দিয়ে Function Expression

JAVASCRIPT

CODE

```
sayHello();

var sayHello = function () {
  console.log("Hello!");
};
```

এখানে সাধারণত:

```
TypeError: sayHello is not a function
```

কারণ conceptually:

JAVASCRIPT

CODE

```
var sayHello;

sayHello();

sayHello = function () {
  console.log("Hello!");
};
```

Call করার সময় `sayHello` -এর value এখনো function হয়নি।

7.29 Hoisting Summary

var

JAVASCRIPT

VAR

```
console.log(x);
var x = 10;
```

Output:

```
undefined
```

let

JAVASCRIPT

LET

```
console.log(x);
let x = 10;
```

Output:

```
ReferenceError
```

const

JAVASCRIPT

CONST

```
console.log(x);
const x = 10;
```

Output:

```
ReferenceError
```

Function Declaration

JAVASCRIPT

FUNCTION DECLARATION

```
hello();

function hello() {
  console.log("Hello");
}
```

Output:

```
Hello
```

7.30 Declaration বনাম Initialization

এই দুইটা একই জিনিস নয়।

JAVASCRIPT

CODE

```
let age = 23;
```

এখানে:

```
let age
```

= Declaration

আর:

```
age = 23
```

= Initialization/Assignment

সহজভাবে:

Core Rule: Declaration = variable-এর নাম তৈরি করা Initialization = প্রথম value দেওয়া

PART 04 — REDECLARATION, REASSIGNMENT & MUTABILITY (7.31 – 7.38)

7.31 Hoisting-এর সময় var

JAVASCRIPT

CODE

```
var age = 23;
```

Conceptually:

JAVASCRIPT

CODE

```
var age; // declaration  
  
age = 23; // initialization
```

Hoisting-এর কারণে declaration অংশ আগে setup হয়।

7.32 var Redeclaration

JAVASCRIPT

CODE

```
var x = 10;  
  
var x = 20;  
  
console.log(x);
```

OUTPUT

```
20
```

var একই scope-এ redeclare করতে পারে।

7.33 let Redeclaration

JAVASCRIPT

CODE

```
let x = 10;  
  
let x = 20;
```

OUTPUT

```
SyntaxError
```

একই scope-এ let redeclare করা যায় না।

7.34 `const` Redeclaration

JAVASCRIPT

CODE

```
const x = 10;  
  
const x = 20;
```

এটিও:

```
SyntaxError
```

7.35 `let` Reassignment

JAVASCRIPT

CODE

```
let age = 23;  
  
age = 24;  
  
console.log(age);
```

OUTPUT

```
24
```

`let` reassign করা যায়।

7.36 `const` Reassignment

JAVASCRIPT

CODE

```
const age = 23;  
  
age = 24;
```

OUTPUT

```
TypeError
```

কারণ `const` binding reassign করা যায় না।

7.37 কিন্তু `const` Object পরিবর্তন করা যায় কেন?

এটা খুব গুরুত্বপূর্ণ।

JAVASCRIPT

CODE

```
const user = {  
  name: "Ripon"  
};  
  
user.name = "Shariar";  
  
console.log(user.name);
```

OUTPUT

```
Shariar
```

কারণ `const` object-এর reference পরিবর্তন করতে দেয় না, কিন্তু object-এর property পরিবর্তন করা যায়।

7.38 Example

এটা করা যাবে না:

JAVASCRIPT

CODE

```
const user = {  
  name: "Ripon"  
};  
  
user = {  
  name: "Sharir"  
};
```

কারণ পুরো reference পরিবর্তন করা হচ্ছে।

কিন্তু:

JAVASCRIPT

CODE

```
user.name = "Sharir";
```

allowed!

PART 05 — SCOPE & HOISTING COMBINED, GOLDEN RULES & BEST PRACTICES (7.39 – 7.47)

7.39 Global Scope बनाम Block Scope

JAVASCRIPT

CODE

```
let name = "Global";  
  
if (true) {  
  let name = "Block";  
  console.log(name);  
}  
  
console.log(name);
```

OUTPUT

Block Global

7.40 Function Scope बनाम Block Scope

JAVASCRIPT

CODE

```
function test() {  
  if (true) {  
    var a = 10;  
    let b = 20;  
  }  
  
  console.log(a);  
  console.log(b);  
}  
  
test();
```

OUTPUT

10 ReferenceError

কারণ:

```
var → function scoped  
let → block scoped
```

7.41 Scope + Hoisting একসাথে

এখন একটি গুরুত্বপূর্ণ example:

JAVASCRIPT

CODE

```
var x = 10;

function test() {

  console.log(x);

  var x = 20;

  console.log(x);
}

test();
```

অনেকে ভাবতে পারে প্রথম `console.log(x)` এ 10 আসবে।

কিন্তু output:

```
undefined
20
```

কেন?

কারণ function-এর ভিতরের:

JAVASCRIPT

CODE

```
var x;
```

hoisted হয়ে যায়।

Conceptually:

JAVASCRIPT

CODE

```
var x = 10;

function test() {

  var x;

  console.log(x);

  x = 20;

  console.log(x);
}

test();
```

7.42 খুব গুরুত্বপূর্ণ Interview Question

Question:

Output কী হবে?

JAVASCRIPT

QUESTION:

```
var x = 10;

function test() {

  console.log(x);

  var x = 20;
}

test();
```

Answer:

undefined

কারণ:

Function scope-এর local `x` hoisted হয়েছে।

7.43 `let` দিয়ে একই Example

JAVASCRIPT

CODE

```
let x = 10;

function test() {

  console.log(x);

  let x = 20;
}

test();
```

OUTPUT

ReferenceError

কারণ local `x` TDZ-এর মধ্যে রয়েছে।

7.44 Scope-এর Golden Rule 🏆

মনে রাখবে:

```
Global
  ↓
Outer Function
  ↓
Inner Function
  ↓
Block
```

Inner scope সাধারণত outer scope access করতে পারে।

কিন্তু outer scope inner scope-এর variable সরাসরি access করতে পারে না।

7.45 Scope-এর Real-life Full Example

ধরো একটি e-commerce website:

JAVASCRIPT

CODE

```
let shopName = "Tech Shop";

function customer() {

  let customerName = "Ripon";

  function order() {

    let product = "Laptop";

    console.log(shopName);
    console.log(customerName);
    console.log(product);

  }

  order();

}

customer();
```

OUTPUT

Tech Shop Ripon Laptop

এখানে:

Global shopName customer() customerName order() product

order() সব outer scope access করতে পারে।

7.46 কেন Scope গুরুত্বপূর্ণ?

Real-world application-এ scope ব্যবহার হয়:

1. Data protection

সব variable global না রেখে নির্দিষ্ট জায়গায় রাখা যায়।

2. Name conflict কমানো

JAVASCRIPT

2. NAME CONFLICT কমানো

```
let name = "A";
```

অন্য scope-এ আবার:

JAVASCRIPT

2. NAME CONFLICT কমানো

```
let name = "B";
```

থাকতে পারে।

3. Code maintainability

বড় project-এ কোন variable কোথায় ব্যবহার করা যাবে তা পরিষ্কার থাকে।

4. Security/encapsulation

Internal data বাইরে expose না করে রাখা যায়।

7.47 Best Practice

Modern JavaScript-এ সাধারণত:

JAVASCRIPT

CODE

```
const
```

প্রথম পছন্দ।

Value পরিবর্তন করতে হলে:

JAVASCRIPT

CODE

```
let
```

আর:

JAVASCRIPT

CODE

```
var
```

সাধারণত avoid করা হয় modern code-এ।

EXAMPLE

✗ পুরোনো style:

JAVASCRIPT

EXAMPLE

```
var username = "Ripon";
```

✓ Modern:

JAVASCRIPT

EXAMPLE

```
const username = "Ripon";
```

যদি value পরিবর্তন হয়:

JAVASCRIPT

EXAMPLE

```
let score = 0;
```

```
score = 10;
```

PART 06 — COMPREHENSIVE CHEAT SHEET, MUST KNOW & PRACTICE LAB

CHEAT SHEET

🗨 Quick Cheat Sheet

Concept	সহজ অর্থ
Scope	Variable কোথায় accessible
Global Scope	গ্রায় পুরো script/module-এর top-level scope
Function Scope	Function-এর ভিতরের scope
Block Scope	<code>{ }</code> block-এর scope
Scope Chain	এক scope থেকে outer scope-এ variable খোঁজা
Lexical Scope	Code কোথায় লেখা হয়েছে তার ভিত্তিতে scope
Shadowing	Inner variable outer variable-কে আড়াল করে
Hoisting	Execution-এর আগে declaration setup
TDZ	<code>let / const</code> declaration-এর আগের inaccessible period
<code>var</code>	Function scoped
<code>let</code>	Block scoped
<code>const</code>	Block scoped + reassignment করা যায় না

MUST KNOW 🔥 Chapter 7 — Must Know

এই chapter থেকে অবশ্যই এগুলো ভালোভাবে জানতে হবে:

1. Scope কী?
2. Global Scope
3. Function Scope
4. Block Scope
5. var vs let vs const
6. Nested Scope
7. Scope Chain
8. Lexical Scope
9. Shadowing
10. Hoisting
11. var Hoisting
12. let/const + TDZ
13. Function Hoisting
14. Function Declaration vs Function Expression
15. Declaration vs Initialization

Practice 1

Output की হবে?

JAVASCRIPT

CODE

```
let x = 10;

function test() {
  let x = 20;
  console.log(x);
}

test();
console.log(x);
```

Practice 2

Output की হবে?

JAVASCRIPT

CODE

```
var x = 10;

if (true) {
  var x = 20;
}

console.log(x);
```

Practice 3

Output की হবে?

JAVASCRIPT

CODE

```
let x = 10;

if (true) {
  let x = 20;
  console.log(x);
}

console.log(x);
```

Practice 4

Output की হবে?

JAVASCRIPT

CODE

```
console.log(a);

var a = 100;
```

Practice 5

Output की হবে?

JAVASCRIPT

CODE

```
console.log(a);

let a = 100;
```

Practice 6

Output কী হবে?

JAVASCRIPT

CODE

```
hello();

function hello() {
  console.log("Hello JavaScript");
}
```

Practice 7

Output কী হবে?

JAVASCRIPT

CODE

```
var x = 10;

function test() {
  console.log(x);

  var x = 20;
}

test();
```

Practice 8

এই code-এ error কেন হচ্ছে?

JAVASCRIPT

CODE

```
function test() {
  let name = "Ripon";
}

test();

console.log(name);
```

SUMMARY 🎯 Chapter 7-এর মূল ধারণা এক লাইনে

Core Rule: Scope বলে variable কোথায় ব্যবহার করা যাবে, আর Hoisting বোঝায় JavaScript execution-এর আগে declaration কীভাবে setup করে।

পরের chapter হবে:

NEXT CHAPTER 🚀 Chapter 8 Preview — Strings

String তৈরি → indexing → length → methods → slicing → searching → replacing → splitting → joining → template literals → escape characters → Unicode → string immutability → practical projects.